VozGuard / EchoTrap — Desarrollo exhaustivo y accionable (The Next Craft · Track 02 — Out of the Box)

> **Cómo leer este documento:** distingue **[HECHO VERIFICADO]** (fuente pública real citada) de **[INFERENCIA/PROPUESTA]** (decisión de diseño del equipo, no un hecho). No se inventan cifras, papers, empresas ni precios. Donde una cifra circula secundariamente y es dudosa, se marca explícitamente.

---

0. Contexto del hackathon (verificado)

[HECHO VERIFICADO] The Next Craft es el hackathon presencial de Crafter Station: **12 horas** (09:00–21:00, sábado 29 ago 2026), simultáneo en Lima, Bogotá, Guatemala, Arequipa y El Salvador; 300 hackers; equipos de 3–5. Code freeze 20:00; **demos a las 20:15, 3 minutos por equipo, "live product"**. Premios: $1,000 / $500 cash + créditos de partners. El jurado evalúa **con demo en vivo + pitch**. La web pública lista como tools/sponsors: **Convex, Clerk, Tavily, ElevenLabs, Exa, Vapi, Apify, Cursor, Replit, n8n**. Cada participante recibe $50 en créditos Vapi, créditos ElevenLabs, 1,000 Tavily, $50 Exa, $50 Apify, $20 Cursor, Replit Core + $20, y un mes de n8n Cloud Pro.

Nota: **MiniMax no aparece en la lista pública de sponsors** de la web, pero el equipo confirma acceso; se trata como herramienta disponible.

TL;DR (respuesta directa a la decisión central)

- **Construyan la versión HÍBRIDA ACOTADA**, no el ecosistema completo ni el detector puro: un **detector de voz clonada en tiempo real** (núcleo sólido y de bajo riesgo, que es la idea del compañero) **+ una capa "EchoTrap" honeypot ligera y local** que se activa al detectar . La demo se hace con **dos teléfonos por altavoz**, sin telefonía real (Vapi) entrante. Esto maximiza el WOW y el "problema real resuelto" sin asumir el riesgo nº1 de demos rotas (telefonía/webhooks).
- **Sí es técnicamente factible** detectar voz clonada en tiempo real con APIs/modelos reales existentes hoy (Resemble Detect, Reality Defender, AI or Not, o un modelo wav2vec2 en el navegador). **Backend sí** (delgado: Convex, para ocultar API keys + realtime). **PWA sí** (instalable, micrófono en primer plano), **pero descarten el monitoreo pasivo 24/7 en background: las PWAs no lo permiten de forma fiable, sobre todo en iOS.**
- **Diferencial honesto:** las piezas ya existen sueltas (Pindrop, Resemble, Hiya detectan; Jolly Roger hace honeypot), pero **nadie las combina para el consumidor común de LATAM** en una PWA localizada con detección + honeypot + alerta a la red familiar. Ese es el hueco real; no digan "nadie lo ha hecho".

1. Solución al problema — paso a paso

Problema: estafas por clonación de voz (voice cloning / vishing). Un atacante clona la voz de un familiar o directivo con segundos de audio y llama para pedir dinero urgente ("secuestro virtual", el "cuento del tío" moderno, o fraude BEC contra empresas).

Qué detecta VozGuard/EchoTrap: si la voz que se escucha en una llamada es **sintética (clonada por IA)** o **humana real (bonafide)**, y en la versión completa además **dilata y neutraliza** al estafador.

Cómo lo detecta (pipeline):

1. Captura de audio del micrófono (o del altavoz del otro teléfono en la demo).
2. **VAD ligero** (Silero VAD vía onnxruntime-web) filtra silencio y solo procesa segmentos con voz — ahorra CPU/batería.
3. Se acumulan ventanas de ~3–5 s de voz.
4. Se envían a un **clasificador anti-spoofing / deepfake de audio** (API externa o modelo wav2vec2 en el navegador).
5. Se obtiene un **score de probabilidad de síntesis** (0–100) + veredicto (real / probablemente sintética / no concluyente).
6. La UI muestra un **semáforo en tiempo real**: humano / dudoso / voz clonada.
7. (Versión completa) Si es , un **agente honeypot (EchoTrap)** toma la llamada, la dilata con conversación y alerta al usuario/familia.

Por qué es mejor que las alternativas actuales:

- **Apps anti-spam (Truecaller, Hiya básico)** bloquean por **número/reputación**, no analizan el **contenido de la voz**. Un número legítimo secuestrado o *spoofeado* pasa el filtro.
- **Verificación manual** ("palabra clave familiar", "cuelga y vuelve a llamar") depende de que la víctima —bajo pánico— recuerde hacerlo. En casi todos los casos evitados que documenta la prensa, "alguien verificó"; el problema es que la mayoría no lo hace bajo presión emocional.
- **VozGuard actúa sobre la señal acústica** (artefactos de síntesis imperceptibles al oído humano), que es exactamente donde falla el juicio humano. **[HECHO VERIFICADO]** El meta-análisis de Diel et al. (2024, *Computers in Human Behavior Reports*), sobre 56 estudios y 86,155 participantes, halló que **la precisión humana total detectando deepfakes fue del 55.54%** (IC 95% [48.87, 62.10]) — con el límite inferior tocando el azar. La máquina supera consistentemente al oído.

---

2. Evaluación crítica de la idea simplificada del compañero

Idea: solo una app que detecta si una voz es clonada, y para la demo simular la llamada con **dos teléfonos físicos** (uno reproduce por altavoz una voz clonada con MiniMax/ElevenLabs; el otro corre la app que escucha y clasifica). Sin Vapi/telefonía real ni honeypot.

¿Viable en 6–10 h?

[INFERENCIA] Sí, es la opción **más viable** de las tres. Elimina la parte más frágil (telefonía real, orquestación Vapi, webhooks públicos) y concentra el esfuerzo en lo que produce el "wow" verificable: el veredicto en vivo.

Qué gana y qué pierde el equipo simplificando así

Gana:

- **Riesgo de fallo en vivo mucho menor:** no depende de PSTN/SIP, números, latencia de operador ni webhooks. Es la causa nº1 de demos rotas en hackathons.
- **Entendible en 10 segundos:** "hablo yo → ; suena el clon → ". Cumple el criterio de demo visual del jurado.
- **Tiempo:** deja horas para pulir UI y ensayar el pitch.

Pierde:

- **Factor WOW:** un semáforo detector es correcto pero menos memorable que "la IA atiende al estafador y lo hace perder el tiempo".
- **"Resuelve un problema real" a medias:** detectar está bien, pero no *protege* activamente; el usuario asustado aún debe actuar.
- **Difícil de copiar:** un detector puro es más fácil de replicar (ya existen Hiya, Resemble, etc.), lo que resta al criterio de "difícil de copiar".

¿Es técnicamente factible detectar voz clonada en tiempo real con precisión aceptable? — Herramientas/APIs REALES

[HECHO VERIFICADO] Sí. Existen múltiples herramientas reales con API pública y accesibles para desarrolladores en poco tiempo:

Herramienta	¿API self-serve?	Free tier	Precio	Exactitud (fuente)
Pindrop Pulse	No (enterprise)	No	Enterprise	Detecta en ~2 s; **>90%** en deepfakes no vistos; hasta **99.4%** con suite completa (<1% FP). **[HECHO VERIFICADO]** En evaluación **independiente de NPR** (84 clips de 5–8 s), Pindrop acertó **81/84 = 96.4%**, superando al competidor más cercano (47/84, 56%) por 40 puntos.

Herramienta	¿API self-serve?	Free tier	Precio	Exactitud (fuente)
Resemble AI Detect (DETECT-3B Omni / 2B)	**Sí**	**$0/mes, sin tarjeta, créditos que no expiran** (demo pública: 2 envíos gratis)	**$0.035/segundo de audio** (Flex); $0.015/s en Team/Business	98.1% (Podonos) / 99.5% homepage / 98.1% ASVspoof 2021 — **benchmarks del propio proveedor, no independientes**
Reality Defender RealAPI	**Sí**	**50 escaneos de audio/imagen gratis/mes**; SDK "2 líneas" (Python/TS/Java/Go/Rust)	Pago usage-based / enterprise (no público)	**No publica cifras de exactitud**
AI or Not	**Sí** (key incluida en free)	**$5 en créditos gratis**	Pro $5/mes habilita detección de voz	Vendedor-reportado
DeepfakeDetector.ai	Sí (desde Starter)	**50 detecciones gratis/mes** (audio+video+imagen)	Starter $49/mes (1,000), Business $199, Enterprise $599	Alta autoreportada
aivoicedetector.com	Sí (planes pago)	Checker público gratis, sin cuenta	~$12.99/mes Pro	~98% autoreportado; JSON <0.5 s con modelo atribuido
ElevenLabs AI Speech Classifier / Audio Detector	**Sí** (y web, gratis)	**Gratis**	Gratis	**Solo detecta audio generado por ElevenLabs**, y no el modelo Eleven v3 — NO es detector general
Modelo en navegador (wav2vec2)	N/A (open source)	Gratis	Gratis	**[HECHO VERIFICADO]** `MelodyMachine/Deepfake-audio-detection-V2` (94.6M params, accuracy 0.9973 en su set); un paper de arXiv corrió un Wav2Vec2-300m **enteramente en navegador con un solo core** (98.1% ASVspoof 2021)

Corrección importante de dato circulante: el número "$0.001 por segundo" de Resemble que aparece en algunos blogs **es incorrecto**; la página oficial de precios de Resemble lista **$0.035/segundo** para detección de audio (Flex). El "$0.001/s" parece confusión con su tarifa de *síntesis* TTS. **No repitan "$0.001/s" como hecho.**

Advertencia de honestidad crítica [HECHO VERIFICADO]: la exactitud **cae** con audio comprimido, recortado, mezclado con música, **reproducido por altavoz o pasado por línea telefónica**, o generado por un modelo que el detector no vio. Esto es directamente relevante al plan de "dos teléfonos por altavoz": el doble paso D/A→A/D degrada la señal y **un detector puede confundir "voz real por altavoz malo" con "sintética"**. Los benchmarks altos suelen medirse sobre audio limpio. Hay que ensayar con los teléfonos reales y ajustar umbral.

Recomendación final: versión HÍBRIDA ACOTADA

[INFERENCIA/PROPUESTA] Construir el **detector en tiempo real** (base de la idea del compañero, núcleo sólido y de bajo riesgo) **+ una capa "EchoTrap" honeypot ligera y local**: al detectar , la app **no llama por PSTN**, sino que **lanza un agente conversacional (ElevenLabs Agents o Vapi en modo web)** que reproduce respuestas dilatorias y muestra en pantalla la transcripción del estafador siendo entretenido. Así se gana el WOW y el "problema real resuelto" sin asumir el riesgo de integrar **telefonía entrante real** en 10 h. Si a mitad del hackathon el honeypot se complica, **fallback a detector-only**, que ya es demo-completa.

Justificación de ganabilidad: encaja literalmente con Track 02 ("una interfaz de voz que responde al instante", "algo generativo y en vivo que reacciona a quien lo usa"); es entendible en 10 s; el clip clonado + el en vivo generan reacción; y la combinación es más difícil de copiar que un detector puro.

---

3. Arquitectura y flujo completo del producto final recomendado

Recomendado (híbrido, riesgo controlado):

```
Usuario (celular con la PWA abierta)
  │  getUserMedia() → MediaStream
  ▼
Frontend (Next.js + React, HTTPS obligatorio)
  │  AudioWorklet → Silero VAD (onnxruntime-web)   [solo procesa cuando hay voz]
  │  buffers de 3–5 s (PCM 16 kHz mono, downsampled)
  ▼
Detección
  ├─ Opción A (recomendada demo): POST del buffer → Convex action → API detección (Resemble/R
  └─ Opción B (edge/privacy): wav2vec2 ONNX en el navegador (transformers.js)
  ▼
Score + veredicto → Convex (persistencia realtime del historial de la llamada)
  ▼
UI: semáforo en vivo + timeline de scores + (si  ) botón "Activar EchoTrap"
  ▼
[EchoTrap honeypot opcional] ElevenLabs/Vapi Agent reproduce respuestas dilatorias;
   transcripción en vivo (STT) se muestra en pantalla vía Convex realtime;
   alerta a familiar de confianza (n8n → WhatsApp/SMS)
```

Flujo con telefonía real (Vapi) — solo si sobra tiempo, NO recomendado para la demo:

```
Estafador → número Vapi entrante → Vapi orquesta STT+LLM+TTS (honeypot)
   → audio de la llamada se streamea a la API de detección
   → veredicto + transcripción → Convex → dashboard del usuario + push/alerta
```

[INFERENCIA] Recomendación: NO usar telefonía entrante real en la demo. Es la fuente de fallo más alta (números, verificación de operador, latencia de carrier, webhooks públicos con HTTPS). Usar el **flujo de dos teléfonos + honeypot local** como demo principal, y **mencionar Vapi como "camino a producción"** en el pitch.

Flujo demo de dos teléfonos (plan principal):

```
Teléfono A (atacante simulado): reproduce por altavoz un audio clonado
   (voz generada previamente con MiniMax voice-clone o ElevenLabs Instant Voice Clone)
Teléfono B: corre la PWA VozGuard, micrófono escuchando
   → VAD → detección →   en <5 s → (opcional) activa EchoTrap honeypot
```

---

4. ¿Se necesita backend o no?

[INFERENCIA, con base técnica verificada] Sí, se recomienda un backend mínimo pero delgado. Razones:

1. **Seguridad de API keys:** las APIs de detección (Resemble, Reality Defender) y de voz (MiniMax, ElevenLabs, Vapi) **no deben exponer su key en el cliente**. Se necesita un proxy server-side. Convex functions o Next.js Route Handlers cumplen esto.
2. **Persistencia realtime del veredicto/transcripción: [HECHO VERIFICADO]** Convex ofrece base de datos + realtime (hook `useQuery`) sin montar WebSockets propios — ideal para el timeline en vivo y para sincronizar el dashboard con el honeypot.
3. **Procesamiento pesado:** si se usa detección por API, el backend solo hace de proxy (ligero). Si se usa el modelo en navegador (transformers.js/ONNX), esa inferencia es 100% cliente y **no** necesita backend.

¿Puede ser 100% cliente (edge/browser)? [HECHO VERIFICADO] Técnicamente sí para la *inferencia* (existe el precedente del plugin Chrome con Wav2Vec2 corriendo en navegador con un solo core). Pero seguirías necesitando ocultar keys si usas cualquier API de pago, y perderías la persistencia realtime del historial.

Recomendación final: Frontend Next.js (captura + VAD + UI) + Convex como backend realtime/proxy. Modelo de detección: **API server-side por defecto**; modelo en navegador como **plan B offline** y como diferenciador "privacy-first" (el audio nunca sale del dispositivo).

---

5. ¿PWA sí o no?

Criterio	PWA	Nativa (Android/iOS)	Web app normal
Micrófono en tiempo real	getUserMedia (HTTPS)		
Interceptar llamadas telefónicas reales	(ni PWA ni web pueden)	⚠ Android limitado, iOS	
Tiempo de dev en hackathon	rápido (una sola base web)	lento	el más rápido
Instalable en el celular para la demo	"Add to Home Screen"	store (lento de publicar)	⚠ solo navegador
Offline	service worker (limitado en iOS)		
Background persistente	muy limitado (ver §7)		

[HECHO VERIFICADO] Ni PWA ni web pueden acceder al **stream de una llamada de voz nativa (GSM/VoLTE)** del teléfono — ni Android ni iOS lo exponen al navegador. Solo pueden escuchar por **micrófono** (p.ej. el altavoz del otro teléfono). Esto **valida el enfoque de dos teléfonos** y descarta técnicamente "interceptar la llamada real desde el navegador".

Recomendación final: PWA (Next.js + manifest + service worker). Es la mejor relación esfuerzo/impacto para el hackathon: se instala en el celular para la demo, corre el micrófono en primer plano y es una sola base de código. **No** intentar app nativa (no hay tiempo) ni depender de background persistente. Diseñar el uso como **activación manual / en primer plano** ("abre VozGuard cuando entre una llamada sospechosa"), no como monitor pasivo 24/7.

---

6. Uso de MiniMax y demás herramientas del sponsor

MiniMax [HECHO VERIFICADO]: plataforma de generación (texto, imagen, video Hailuo, voz, música). Modelo de voz **actual 2026: familia Speech 2.8 (HD y Turbo/Flash) + Voice Design**; **speech-2.6 y speech-02 quedan como "legacy"** pero siguen operativos (IDs `speech-2.6-hd`, `speech-02-hd` funcionan). Clonación vía **voice-cloning** (referencia ≥10 s, formatos mp3/m4a/wav) que devuelve un `voice_id` reutilizable. Precio pay-as-you-go: **$100 por millón de caracteres (HD), $60 por millón (Turbo)**; Rapid Voice Cloning ~$1.50/voz; Voice Design ~$3/voz. **MiniMax NO tiene API de detección de deepfake — solo genera/clona** (confirmado: no existe producto de verificación documentado a ago 2026).

→ **Rol de MiniMax en el proyecto: generar la voz clonada de prueba** para la demo — clonar una voz **de consentimiento del equipo** y hacerla decir el guion del "estafador". Es el generador del "ataque" que el detector debe cazar. Speech-02/2.6/2.8 Turbo (sub-segundo) sirve también si se quiere honeypot generativo.

Roles del resto (propuesta):

- **ElevenLabs [HECHO VERIFICADO: gratis + API]:** (a) segunda voz clonada alternativa para la demo (Instant Voice Cloning); (b) **AI Speech Classifier** como *segunda señal* de detección (recordar: solo detecta audio de ElevenLabs); (c) STT (**Scribe v2 Realtime, <150 ms**) para transcribir al estafador en el honeypot; (d) TTS del agente EchoTrap.
- **Vapi [HECHO VERIFICADO: ~$0.05/min orquestación + BYO stack]:** camino a producción para telefonía real y/o motor del agente honeypot conversacional. Los $50 de crédito/participante alcanzan para la demo.
- **Convex [HECHO VERIFICADO]:** backend realtime + DB + file storage + proxy de keys. Núcleo del timeline en vivo. Es sponsor principal — usarlo suma puntos de "buen uso de stack".
- **Clerk:** autenticación (login, perfil, "mis contactos de confianza").
- **Tavily / Exa:** búsqueda web para enriquecer el dashboard (verificar número entrante contra reportes de fraude; poblar datos del pitch). Rol secundario.
- **Apify:** scraping de listas públicas de números reportados como spam (enriquecimiento). Rol secundario/opcional.

- **n8n Cloud Pro:** automatización de alertas (al detectar → WhatsApp/email/SMS a un familiar de confianza). Buen "wow" de flujo.
- **Cursor / Replit:** entorno de desarrollo y despliegue rápido.

---

7. App en segundo plano con bajo consumo — recomendaciones reales y honestas

[HECHO VERIFICADO] Las PWAs NO pueden ejecutar audio/micrófono en segundo plano de forma persistente y fiable, especialmente en iOS. Hechos concretos:

- iOS limita severamente la ejecución en background de web apps para ahorrar batería; el **Background Sync API no tiene soporte fiable en iOS**; los service workers en iOS sirven sobre todo para **caché y push**, no para tareas de propósito general en background.
- En la UE, Apple llegó a remover las home-screen web apps (por el DMA) en 2024, afectando modo standalone y push.
- Android es algo más permisivo pero también restringe el background de PWAs.
- Un equipo (Prototyp) documentó que ni la reproducción de audio en background funcionó en iOS sin envolver la PWA en un shell nativo (Cordova).

Conclusión honesta: un "monitor pasivo que escucha todas las llamadas 24/7 en background" **no es realista como PWA**. Diséñenlo como **activación en primer plano**.

Recomendaciones técnicas concretas para eficiencia (cuando la app está activa):

1. **VAD ligero antes del modelo pesado:** Silero VAD (ONNX, ~2 MB, 8/16 kHz) en un **AudioWorklet**; solo cuando detecta voz (`onSpeechStart`) se dispara la clasificación. Evita gastar CPU/red en silencio.
2. **Procesamiento por lotes, no streaming continuo:** clasificar ventanas de 3–5 s o al final de cada turno (`onSpeechEnd`), no cada frame. (Además reduce costo de API: ver §10.)
3. **AudioWorklet en vez del obsoleto ScriptProcessorNode:** corre en el render thread de audio, más eficiente.
4. **Downsample a 16 kHz mono** antes de inferir (lo que esperan los modelos anti-spoofing) — menos datos, menos cómputo.
5. **Modelo en navegador con WASM/WebGPU + un solo core** (precedente del plugin Chrome) para no depender de red y reducir latencia/energía; o API con requests espaciados.
6. **Wake Lock API** solo mientras dura la llamada activa, liberándolo al terminar, para no drenar batería.
7. **Activación por notificación/manual:** el usuario abre VozGuard (o toca una push) cuando entra la llamada sospechosa, en vez de monitoreo continuo. Es el patrón honesto y viable.
8. Si a futuro se requiere background real → **app nativa** (Android `Service` / `ConnectionService` ; iOS `CallKit` con sus límites) — fuera del alcance del hackathon.

---

8. Dónde desplegar (deployment)

Comparación:

- **Vercel + Convex [HECHO VERIFICADO]:** integración oficial (Convex en el Vercel Marketplace; template Next.js+Convex listo en <2 min; inyecta las env vars automáticamente al build). HTTPS automático (requisito de `getUserMedia`). Free tier generoso para hackathon. El realtime lo maneja Convex — **no necesitas montar WebSockets propios**. Menor fricción.
- **Cloudflare Pages/Workers:** excelente en edge y costos, HTTPS automático; pero la integración con Convex y con Next.js (App Router/SSR) es más artesanal y con más aristas para WebSockets/realtime bajo presión de tiempo. Mejor para 100% edge sin backend estatal.
- **Netlify:** viable (la doc de Convex lo menciona), similar a Vercel pero sin la integración nativa tan pulida.

Recomendación final de arquitectura de despliegue:

- **Frontend + Route Handlers: Vercel** (Next.js).
- **Backend realtime + DB + proxy de keys: Convex** (`npx convex deploy --cmd 'next build'`).
- **HTTPS:** cubierto por Vercel (**imprescindible** para acceso a micrófono).
- **Base de datos:** Convex (integrada). Todo en free tier alcanza para el hackathon.

---

9. Librerías y dependencias completas

> Las versiones cambian rápido; confirmar con `npm view <pkg> version` el día del hackathon. Abajo, **nombres reales de paquetes** y familias de versión verificadas.

Frontend / framework

- `next` (Next.js 15.x), `react`, `react-dom`
- `tailwindcss`, `shadcn/ui` (o `@radix-ui/*`) para UI rápida
- `lucide-react` (iconos), `framer-motion` (animación del semáforo)

Captura y procesamiento de audio

- Web Audio API nativa (`AudioContext`, `AudioWorkletNode`) — sin paquete
- `@ricky0123/vad-web` (VAD en navegador; usa Silero VAD) + `onnxruntime-web`
  - **[HECHO VERIFICADO]** los ejemplos oficiales usan `onnxruntime-web@1.14.0` y `@ricky0123/vad-web` en versiones 0.0.7 / 0.0.19 / 0.0.22 — instalar la última estable. Sirve los archivos `silero_vad_v5.onnx` y `vad.worklet.bundle.min.js`.
- `@ricky0123/vad-react` si se quiere hook React

Detección de voz clonada / deepfake

- **Vía API (recomendado):** `fetch` nativo contra Resemble Detect (`docs.resemble.ai`) o el **SDK oficial de Reality Defender** (confirmar nombre exacto del paquete npm en su doc; tienen SDK

TS). AI or Not como alternativa barata.
- **Vía modelo en navegador (plan B):** `@huggingface/transformers` (Transformers.js) cargando un modelo wav2vec2 anti-spoofing (p.ej. `MelodyMachine/Deepfake-audio-detection-V2`) exportado a ONNX; runtime `onnxruntime-web` .

Voz (generación/clonación para la demo y honeypot)

- MiniMax: SDK oficial `@minimax/minimax-mcp-js` o REST directo; o vía `replicate` (`minimax/voice-cloning` , `minimax/speech-02-turbo`) / `@fal-ai/client`
- ElevenLabs: `@elevenlabs/elevenlabs-js` (TTS/STT/voice clone)
- Vapi: `@vapi-ai/web` (cliente web) / `@vapi-ai/server-sdk`

Backend / DB / realtime

- `convex` (functions, DB, realtime `useQuery` , file storage)
- `@clerk/nextjs` (auth) + integración Convex-Clerk

Automatización / alertas (opcional)

- n8n Cloud (no es npm; se integra por webhook disparado desde una Convex action)

Utilidades

- `zod` (validación), `date-fns`

---

10. Pitch de 3 minutos

[HECHO VERIFICADO — estructura del deck original de Airbnb 2008/2009]: 14 slides en orden **Problema → Solución → Validación de mercado → Tamaño de mercado → Producto → Modelo de negocio → Adopción → Competencia → Ventaja competitiva → Equipo → Prensa → Testimonios → Financiero**. El slide de problema eran **3 afirmaciones claras e indiscutibles centradas en el cliente** (ninguna mencionaba a Airbnb); la solución **"espejaba" cada uno de los tres problemas** (costo → "ahorra dinero", desconexión → "comparte cultura", mercado faltante → "gana dinero hospedando"). Adaptamos ese **estilo narrativo**, no sus datos.

El problema (fuentes reales 2025–2026, LATAM/Perú donde existe)

- **[HECHO VERIFICADO]** El **FBI 2025 Internet Crime Report** (publicado el 6 de abril de 2026), por primera vez en 25 años, desglosó el fraude habilitado por IA: **22,364 quejas y $893,346,472 en pérdidas ajustadas**; el total del año fue de **1,008,597 quejas y ~$20.9 mil millones**. Las estafas de "distress" con voz clonada sumaron >$5M, cifra que el propio FBI marca como subestimada.
- **[HECHO VERIFICADO]** La **FTC** reportó que en 2025 la gente perdió **$3.5 mil millones a estafas de suplantación** (imposter scams), con pérdidas que casi se triplicaron desde 2020; ~1 de cada 3 reportes de fraude fueron de suplantación ($1,000M a suplantadores de empresas/bancos, $920M a suplantadores de gobierno).

- **[HECHO VERIFICADO]** El informe **McAfee "The Artificial Imposter" (mayo 2023, 7,054 personas en 7 países)**: **3 segundos de audio bastan para lograr un 85% de coincidencia** de voz (95% con más entrenamiento); **77% de las víctimas perdió dinero** y 53% de adultos comparte su voz online semanalmente.
- **[HECHO VERIFICADO]** El oído humano no protege: precisión media del **55.54%** detectando deepfakes (Diel et al., 2024, 86,155 participantes).
- **[HECHO VERIFICADO — Perú/LATAM]** El Comercio, La República, ATV y Claro Perú documentan la modalidad de clonación de voz de familiares en Perú ("secuestro virtual"); el caso de la madre de **Milett Figueroa** es un ejemplo mediático nacional. Pindrop (Amit Gupta) explica que "el objetivo no es una réplica perfecta de la voz, sino generar suficiente duda y urgencia para que la víctima actúe sin verificar nada".

La funcionalidad que resuelve

Detectar en tiempo real si la voz de una llamada es clonada y, si lo es, proteger al usuario (semáforo + honeypot que dilata al estafador + alerta a la familia).

La solución (espejo del problema, estilo Airbnb)

- "No puedes confiar en tu oído" → **VozGuard oye por ti** (semáforo en vivo).
- "Actúas bajo pánico" → **veredicto instantáneo en <5 s**.
- "El estafador te presiona" → **EchoTrap lo entretiene y avisa a tu red familiar**.

Factor diferencial (honesto y verificado)

[HECHO VERIFICADO] Sí existen productos de detección: Pindrop (enterprise/call centers), Resemble Detect (API), **Hiya** (extensión Chrome gratuita para consumidor, >99% autoreportado, 1 s de audio), Reality Defender, ElevenLabs classifier. **Y existe** el precedente honeypot: **Jolly Roger Telephone Company** (bots con GPT + clonador de voz que dilatan telemarketers, ~$25/año o $1.99/mes). **Lo que NO está construido como producto de consumo** es la **combinación**: detección en vivo **para el ciudadano común de LATAM** + **honeypot defensivo automático** + **alerta a la red familiar**, en una **PWA instalable y localizada en español**. Ese es el hueco real. → En el pitch, decir "las piezas existen sueltas; nadie las junta para el consumidor LATAM", **no** "nadie lo ha hecho".

Modelo de negocio realista

Freemium consumidor (detección gratis limitada; honeypot + alertas familiares en suscripción **~$2–5/mes**, análogo a Jolly Roger $25/año) + **licencia B2B** a bancos/telcos (API de liveness/detección, como Pindrop/Resemble).

Proyección / escalabilidad

De PWA a app nativa con CallKit/ConnectionService para monitoreo real; API para bancos; alianzas con operadores (Claro/Movistar) para protección a nivel de red en LATAM.

Estimación de costos en producción (cifras públicas reales)

- **Detección:** Resemble Detect **$0.035/segundo** de audio (Flex); Reality Defender 50 gratis/mes luego enterprise; DeepfakeDetector.ai **$49/mes** por 1,000 detecciones; AI or Not Pro **$5/mes**.
- **Voz / honeypot: Vapi ~$0.05/min** orquestación + STT/LLM/TTS (total real de mercado **~$0.13–0.31/min**); **ElevenLabs TTS $0.10/1,000 chars** (v2/v3), **$0.05 Flash**; **MiniMax $100/M chars HD, $60/M Turbo**, clonación ~$1.50/voz.
- **Hosting:** Vercel + Convex free tier para arranque; escalado por uso.
- **Ejemplo de costo por llamada honeypot de 5 min:** si se detecta cada segundo → (5 min × ~$0.20/min Vapi) + (300 s × $0.035) ≈ **$1 + $10.5 = $11.5**. **Optimización clave:** detectar **por lotes** (p.ej. 6 muestras de 5 s) baja la detección a **<$0.25** por llamada → costo total realista ~$1.25/llamada.

Guion de pitch con timing (3 min)

- **0:00–0:25 — Gancho + problema:** reproducir en vivo el clip clonado *"Mamá, tuve un accidente, necesito plata ya"*. "Esta voz es falsa. Perú ya vive esta estafa; en EE.UU. el FBI contó $893M en fraude por IA en 2025."
- **0:25–0:45 — Por qué ahora:** "3 segundos de audio bastan (McAfee); el oído humano acierta solo 55%."
- **0:45–1:45 — Demo en vivo (el corazón):** teléfono A reproduce el clon → teléfono B (VozGuard) marca en <5 s → se activa EchoTrap y en pantalla se ve al "estafador" siendo entretenido + alerta a la familia.
- **1:45–2:15 — Solución/diferencial:** "Detección + honeypot + alerta familiar, en una PWA en español, para el ciudadano LATAM. Las piezas existen; nadie las junta para el consumidor."
- **2:15–2:40 — Negocio + escala:** freemium $2–5/mes + B2B bancos/telcos; costos por API reales y sostenibles.
- **2:40–3:00 — Cierre + Track:** "Track 02, Out of the Box: voz en tiempo real que reacciona. Le devolvemos la confianza al teléfono."

---

11. Flujo de funcionamiento completo (checklist para construir, con timing ~10 h efectivas)

- **Fase 0 (0:00–0:30) Setup:** repo Next.js + Convex + Clerk desde template Vercel; env vars; deploy inicial "hello world" con HTTPS verificado en el celular de demo.
- **Fase 1 (0:30–2:00) Captura + VAD:** `getUserMedia` → AudioWorklet → `@ricky0123/vad-web`; log de `onSpeechStart/onSpeechEnd`; UI base con semáforo dummy.
- **Fase 2 (2:00–4:00) Detección:** conectar API de detección vía **Convex action** (key oculta); mapear score → ; timeline realtime con Convex `useQuery`.
- **Fase 3 (4:00–5:00) Voz de demo:** generar 2–3 clips clonados con **MiniMax** (voz de consentimiento del equipo) + guiones de estafa; guardar en el teléfono A.

- **Fase 4 (5:00–7:00) EchoTrap honeypot (opcional/paralelo):** al , disparar agente ElevenLabs/Vapi con respuestas dilatorias + STT en pantalla + alerta n8n.
- **Fase 5 (7:00–8:30) Pulido:** UI, animaciones del semáforo, copy en español, PWA manifest + service worker + "Add to Home Screen".
- **Fase 6 (8:30–10:00) Ensayo de demo + pitch:** probar los dos teléfonos por altavoz varias veces; medir latencia; grabar video de respaldo; preparar el "modo determinista" de plan B.

---

12. Track del hackathon

[HECHO VERIFICADO] Aplican a **Track 02 — OUT OF THE BOX** ("weird, experimental, unclassifiable — con chispa técnica; real-time, interfaces novedosas, generativo, hardware; la regla: haz que la gente reaccione"). VozGuard/EchoTrap encaja en **dos de sus "starting points" literales**: **"una interfaz de voz que responde al instante"** y **"algo generativo y en vivo que reacciona a quien lo usa"**. Es real-time, provoca reacción inmediata (el clip clonado + el semáforo), tiene chispa técnica (VAD + anti-spoofing + honeypot generativo) y resuelve un problema real y doloroso en LATAM. En el pitch, **nombrar el track explícitamente** en el cierre.

---

13. Elementos adicionales para robustez

Estructura de carpetas (Next.js App Router + Convex + PWA):

```
vozguard/
├ app/
│  ├ layout.tsx  page.tsx            (dashboard)
│  ├ api/detect/route.ts             (proxy detección si no se usa Convex action)
│  └ manifest.ts                     (PWA)
├ components/  (Semaphore, Timeline, EchoTrapPanel, MicButton)
├ lib/  (audio/worklet.ts, vad.ts, downsample.ts, detectClient.ts)
├ public/  (silero_vad_v5.onnx, vad.worklet.bundle.min.js, ort-wasm*, icons/)
├ convex/  (schema.ts, detections.ts, honeypot.ts, alerts.ts, http.ts)
├ workers/ (service-worker.ts)
├ .env.local
└ next.config.ts
```

Variables de entorno necesarias:

```
NEXT_PUBLIC_CONVEX_URL=
CONVEX_DEPLOY_KEY=
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=
CLERK_SECRET_KEY=
RESEMBLE_API_KEY=            # o REALITY_DEFENDER_API_KEY / AIORNOT_API_KEY
MINIMAX_API_KEY=  MINIMAX_GROUP_ID=
ELEVENLABS_API_KEY=
VAPI_API_KEY=                # si honeypot/telefonía
N8N_WEBHOOK_URL=             # alertas
```

Checklist de riesgos (actualizado):

1. **Degradación por altavoz/línea [alto]** → ensayar con los teléfonos reales; ajustar umbral; tener clip pregrabado como fallback.
2. **Falsos positivos** (voz real marcada como clonada) → mostrar el **score**, no solo binario; umbral conservador; frasear como "sospecha", no "certeza".
3. **API rate limits / caída** → cachear; tener el **modelo en navegador como plan B**; probar límites antes de la demo.
4. **Permisos de micrófono / HTTPS** → verificar en el navegador del teléfono de demo con antelación (Chrome/Safari piden HTTPS y gesto de usuario).
5. **iOS PWA background** → **NO prometer monitoreo pasivo**; demo en primer plano.
6. **Latencia** → VAD + lotes; objetivo veredicto **<5 s**.
7. **Créditos** → usar los créditos del hackathon; **evitar detección por segundo continua** en pruebas (costo).

Plan B específico (demo dos teléfonos):

- Ensayar volumen y distancia del altavoz del teléfono A.
- Tener **3 clips clonados listos localmente** (no depender de generar en vivo).
- Grabar un **video de respaldo** de una corrida exitosa por si el micrófono falla en vivo.
- **Modo "demo determinista":** botón oculto que fuerza el flujo →EchoTrap si la red cae.

Consideraciones legales / éticas (importante):

- **[HECHO VERIFICADO]** MiniMax y ElevenLabs exigen **consentimiento/derechos** para clonar una voz; MiniMax documenta consentimiento y borrado del `voice_id`. → **Clonar solo la voz de un miembro del equipo que dé consentimiento explícito**; nunca clonar voces de terceros/famosos, ni siquiera para la demo.
- Grabar/procesar audio de terceros implica privacidad: en la demo, **avisar que se captura audio**; no almacenar audio de asistentes sin permiso. El modo **"privacy-first"** (inferencia en navegador, el audio no sale del dispositivo) es también un **argumento ético de venta**.
- **Etiquetar claramente los clips como "generados por IA"** en el pitch para no inducir a error al jurado.
- **No presentar el detector como infalible:** comunicar sus límites (audio comprimido, por altavoz, modelos no vistos) es honesto y **más creíble** ante el jurado que exagerar precisión.

Recomendaciones finales (staged / accionables)

1. **Ahora (decisión):** comprométanse con la **versión híbrida acotada**: detector en tiempo real + EchoTrap ligero, demo con dos teléfonos, sin telefonía real. Es la mejor combinación de factibilidad (6–10 h) y ganabilidad.
2. **Motor de detección:** empiecen con **API self-serve** (Reality Defender por sus **50 escaneos gratis/mes** y SDK de 2 líneas, o AI or Not por su free tier) para arrancar rápido; dejen preparado el **modelo wav2vec2 en navegador** como plan B/diferenciador privacy-first. **Benchmark que**

cambia la decisión: si en el ensayo con los dos teléfonos la precisión por altavoz cae por debajo de ~70–75%, cambien a otro detector, suban el umbral, o pasen al modo determinista de demo.

3. **Stack:** Next.js (PWA) + Convex (realtime + proxy de keys) + Clerk, desplegado en **Vercel + Convex**. MiniMax para generar los clips de ataque; ElevenLabs/Vapi para EchoTrap; n8n para la alerta familiar.
4. **Umbral de descarte del honeypot:** si a la hora 7 el agente EchoTrap no funciona de forma fiable, **córtenlo** y entreguen el detector-only pulido — ya es una demo ganable.
5. **Pitch:** guion de 3 min con el clip clonado como gancho, la demo en vivo como corazón, y el cierre nombrando el Track 02. Comuniquen límites con honestidad: suma credibilidad.

Caveats (límites de este informe)

- Las **cifras de exactitud** de Resemble (98.1–99.5%) y de varias herramientas son **autoreportadas por el proveedor**; solo el 96.4% de Pindrop tiene validación **independiente** (NPR). Trátenlas como marketing hasta probarlas.
- El dato **"$0.001/s de Resemble" que circula en blogs es incorrecto**; el precio oficial es **$0.035/s** para audio. Reality Defender **no publica** cifras de exactitud ni precio por unidad más allá del free tier.
- Las **versiones exactas de npm** cambian; hay que fijarlas el día del evento (`npm view`).
- Los **precios de APIs** (Vapi, ElevenLabs, MiniMax, detección) son públicos a fecha de este informe pero cambian con frecuencia; reconfirmar antes de proyectar costos en el pitch.
- **MiniMax no está en la lista pública de sponsors** de la web del hackathon; el rol asignado asume que el acceso confirmado por el equipo es válido.
- La degradación por altavoz/telefonía es un **riesgo real y no resuelto** para cualquier detector; el plan B determinista existe justamente por esto.
- Ninguna PWA puede acceder al stream de una llamada nativa ni correr monitoreo de audio persistente en background (sobre todo iOS): el producto es, por diseño, de **activación en primer plano**.